

Sprint 6 Structured Notes: Arrays as Ordered Collections

0. What Sprint 6 Is About

Sprint 6 is about **arrays**.

An array is one of the most important tools in JavaScript because it lets you store **many values in one place**.

In earlier sprints, you worked mostly with single values:

```
const name = "Ada";  
const score = 90;  
const isActive = true;
```

Each variable stores one value.

But many real programs need a list of values:

```
const names = ["Ada", "Grace", "Linus"];  
const scores = [90, 85, 100];  
const tasks = ["read", "code", "review"];
```

That is what arrays are for.

Main goal of Sprint 6

Store ordered data and modify it with built-in array operations.

By the end of Sprint 6, you should understand:

- arrays
- ordered collections
- zero-based indexing
- array length
- reading array items
- updating array items
- missing indexes and `undefined`
- `const` arrays
- `push()`

- `pop()`
 - `unshift()`
 - `shift()`
 - `includes()`
 - `indexOf()`
 - `slice()`
 - `splice()`
 - shallow copies
 - spread syntax `...`
 - one-dimensional arrays
 - two-dimensional arrays
 - array destructuring
 - string reversal with array methods
 - beginner array CRUD patterns
 - common beginner mistakes with arrays
-

Part A: Why Arrays Matter

1. The Problem Arrays Solve

Without arrays, you may write many separate variables:

```
const fruit1 = "apple";  
const fruit2 = "banana";  
const fruit3 = "cherry";
```

This works, but it becomes messy when the list grows.

A cleaner version uses one array:

```
const fruits = ["apple", "banana", "cherry"];
```

Now all related values are stored together.

Simple meaning:

An array is a list of values.

Arrays are useful for:

- lists of names

- lists of scores
- lists of products
- lists of tasks
- lists of messages
- lists of users
- rows and columns of data
- data that will later be displayed on a web page

2. How Sprint 6 Connects to Earlier Sprints

Sprint 6 builds directly on Sprints 1–5.

Earlier Sprint	What You Learned	How It Helps With Arrays
Sprint 1	JavaScript can change page behavior.	Arrays can store data that the page displays.
Sprint 2	Values, types, variables, <code>let</code> , <code>const</code> , and references.	Arrays are values and behave like reference-based objects.
Sprint 3	Strings, indexes, <code>.length</code> , <code>slice()</code> , <code>includes()</code> , and <code>indexOf()</code> .	Arrays also use indexes, <code>.length</code> , <code>slice()</code> , <code>includes()</code> , and <code>indexOf()</code> .
Sprint 4	Numbers, booleans, comparisons, and conditions.	Array code often checks indexes, lengths, and whether items exist.
Sprint 5	Functions, parameters, return values, and scope.	You can write functions that add, update, delete, or read array data.

Sprint 6 is where your code starts managing **collections of data**, not only individual values.

Part B: Array Basics

3. What Is an Array?

An array is an ordered collection of values.

Example:

```
const fruits = ["apple", "banana", "cherry"];
```

This array contains three values.

The values are stored in order:

```
apple comes first  
banana comes second  
cherry comes third
```

JavaScript remembers that order.

4. Array Syntax

Arrays use square brackets:

```
[]
```

Example:

```
const fruits = ["apple", "banana", "cherry"];
```

The values inside the array are called **elements** or **items**.

```
"apple" -> first element  
"banana" -> second element  
"cherry" -> third element
```

Each item is separated by a comma.

5. Arrays Can Store Different Types

An array can store strings:

```
const names = ["Ada", "Grace", "Linus"];
```

An array can store numbers:

```
const scores = [90, 85, 100];
```

An array can store booleans:

```
const answers = [true, false, true];
```

An array can even store mixed values:

```
const mixed = ["Ada", 25, true, null];
```

This is allowed in JavaScript.

But beginner code is usually easier to understand when one array stores one kind of data.

Cleaner:

```
const scores = [90, 85, 100];
```

Less clean:

```
const data = [90, "Ada", true, null];
```

Beginner rule:

Use arrays for related values.
Try to keep the type and purpose consistent.

Part C: Indexes and Reading Array Items

6. Zero-Based Indexing

JavaScript arrays are **zero-indexed**.

Simple meaning:

The first item is at index 0.

Example:

```
const fruits = ["apple", "banana", "cherry"];
```

The indexes are:

```
Index:    0        1        2  
Value: "apple" "banana" "cherry"
```

So:

```
console.log(fruits[0]); // apple  
console.log(fruits[1]); // banana  
console.log(fruits[2]); // cherry
```

This is the same basic idea you saw with strings.

Example with a string:

```
const word = "hello";  
console.log(word[0]); // h
```

Example with an array:

```
const letters = ["h", "e", "l", "l", "o"];  
console.log(letters[0]); // h
```

7. Reading an Array Item

To read an array item, use bracket notation:

```
arrayName[index]
```

Example:

```
const colors = ["red", "green", "blue"];  
  
console.log(colors[0]); // red
```

```
console.log(colors[1]); // green
console.log(colors[2]); // blue
```

Read this:

```
colors[1]
```

as:

```
Get the item at index 1 from the colors array.
```

Because arrays start at 0, index 1 is the second item.

8. Accessing a Missing Index

If you try to access an index that does not exist, JavaScript returns `undefined`.

Example:

```
const fruits = ["apple", "banana", "cherry"];

console.log(fruits[99]); // undefined
```

Why?

The array has only three items:

```
Index 0 -> apple
Index 1 -> banana
Index 2 -> cherry
```

There is no index 99.

So JavaScript gives:

```
undefined
```

This does not automatically crash the program.

It means:

There is no value at that position.

Part D: Array Length

9. What Is `.length`?

`.length` tells you how many items are in an array.

Example:

```
const fruits = ["apple", "banana", "cherry"];

console.log(fruits.length); // 3
```

There are three items, so `.length` is `3`.

10. Length Is Not the Last Index

This is a common beginner mistake.

Example:

```
const fruits = ["apple", "banana", "cherry"];

console.log(fruits.length); // 3
console.log(fruits[3]);     // undefined
```

Why is `fruits[3]` undefined?

Because the indexes are:

```
Index 0 -> apple
Index 1 -> banana
Index 2 -> cherry
```

The length is `3`, but the last index is `2`.

Formula:

```
last index = array.length - 1
```

Correct way to get the last item:

```
const fruits = ["apple", "banana", "cherry"];

const lastIndex = fruits.length - 1;
console.log(fruits[lastIndex]); // cherry
```

Shorter version:

```
console.log(fruits[fruits.length - 1]); // cherry
```

Beginner rule:

```
Length counts items.
Indexes count positions starting from 0.
```

Part E: Updating Array Items

11. Changing an Item by Index

You can update an array item using bracket notation and assignment.

Example:

```
const fruits = ["apple", "banana", "cherry"];

fruits[1] = "blueberry";

console.log(fruits);
// ["apple", "blueberry", "cherry"]
```

What happened?

Index 1 used to contain "banana".
Now index 1 contains "blueberry".

12. `const` Arrays Can Still Be Changed Inside

This works:

```
const fruits = ["apple", "banana"];

fruits[0] = "orange";
fruits.push("cherry");

console.log(fruits);
// ["orange", "banana", "cherry"]
```

This is allowed because `const` does not freeze the array contents.

`const` only prevents the variable from being reassigned.

Allowed:

```
const fruits = ["apple", "banana"];
fruits[0] = "orange";
```

Not allowed:

```
const fruits = ["apple", "banana"];
fruits = ["orange", "grape"]; // error
```

Simple meaning:

```
const protects the variable name.
const does not freeze the inside of the array.
```

This is the same basic reference idea from Sprint 2.

Arrays are objects internally, so variables store references to them.

13. Be Careful With Skipped Indexes

You can assign to an index that is far away:

```
const items = ["a", "b"];

items[5] = "f";

console.log(items);
console.log(items.length);
```

This creates empty positions between index 1 and index 5.

This is usually not good beginner code.

Beginner rule:

```
Do not create large skipped indexes on purpose.
Use push() when adding to the end.
```

Cleaner:

```
const items = ["a", "b"];
items.push("c");
```

Part F: Adding and Removing Items

Arrays have built-in methods that add or remove items.

The four beginner methods are:

```
push()
pop()
unshift()
shift()
```

14. `push()` — Add to the End

`push()` adds one or more items to the end of an array.

Example:

```
const fruits = ["apple", "banana"];

fruits.push("cherry");

console.log(fruits);
// ["apple", "banana", "cherry"]
```

`push()` changes the original array.

It also returns the new length.

Example:

```
const fruits = ["apple", "banana"];

const newLength = fruits.push("cherry");

console.log(newLength); // 3
console.log(fruits);    // ["apple", "banana", "cherry"]
```

Important:

```
const result = fruits.push("date");
```

`result` is the new length, not the array.

15. `pop()` — Remove From the End

`pop()` removes the last item from an array.

Example:

```
const fruits = ["apple", "banana", "cherry"];
```

```
const removed = fruits.pop();

console.log(removed); // cherry
console.log(fruits);  // ["apple", "banana"]
```

`pop()` does two things:

1. Removes the last item.
2. Returns the removed item.

If the array is empty:

```
const items = [];

console.log(items.pop()); // undefined
```

There is no item to remove, so it returns `undefined`.

16. `unshift()` — Add to the Beginning

`unshift()` adds one or more items to the beginning of an array.

Example:

```
const fruits = ["banana", "cherry"];

fruits.unshift("apple");

console.log(fruits);
// ["apple", "banana", "cherry"]
```

`unshift()` changes the original array.

It also returns the new length.

Example:

```
const fruits = ["banana", "cherry"];

const newLength = fruits.unshift("apple");
```

```
console.log(newLength); // 3
console.log(fruits);    // ["apple", "banana", "cherry"]
```

17. `shift()` — Remove From the Beginning

`shift()` removes the first item from an array.

Example:

```
const fruits = ["apple", "banana", "cherry"];

const removed = fruits.shift();

console.log(removed); // apple
console.log(fruits);  // ["banana", "cherry"]
```

`shift()` does two things:

1. Removes the first item.
2. Returns the removed item.

If the array is empty:

```
const items = [];

console.log(items.shift()); // undefined
```

18. Add and Remove Method Summary

Method	Position	Action	Changes Original Array?	Returns
<code>push()</code>	End	Adds item	Yes	New length
<code>pop()</code>	End	Removes item	Yes	Removed item
<code>unshift()</code>	Beginning	Adds item	Yes	New length
<code>shift()</code>	Beginning	Removes item	Yes	Removed item

Simple memory trick:

push/pop work at the end.
unshift/shift work at the beginning.

Part G: Finding Items in Arrays

19. `includes()`

Use `includes()` when you only need a yes-or-no answer.

Example:

```
const fruits = ["apple", "banana", "cherry"];

console.log(fruits.includes("banana")); // true
console.log(fruits.includes("grape"));  // false
```

Simple meaning:

Does this array contain this value?

`includes()` returns a boolean:

true or false

Useful in conditions:

```
const fruits = ["apple", "banana", "cherry"];

if (fruits.includes("banana")) {
  console.log("Banana exists");
}
```

20. `indexOf()`

Use `indexOf()` when you need the position of an item.

Example:

```
const fruits = ["apple", "banana", "cherry"];

console.log(fruits.indexOf("banana")); // 1
console.log(fruits.indexOf("grape"));  // -1
```

Simple meaning:

Where is this value in the array?

If the item exists, `indexOf()` returns its index.

If the item does not exist, `indexOf()` returns `-1`.

21. `includes()` vs `indexOf()`

Method	Returns	Use When
<code>includes()</code>	<code>true</code> or <code>false</code>	You only need to know whether the item exists.
<code>indexOf()</code>	index number or <code>-1</code>	You need the item position.

Example with `includes()`:

```
const fruits = ["apple", "banana", "cherry"];

if (fruits.includes("apple")) {
  console.log("Found apple");
}
```

Example with `indexOf()`:

```
const fruits = ["apple", "banana", "cherry"];

const index = fruits.indexOf("banana");
console.log(index); // 1
```


22. Checking `indexOf()` Correctly

Do not use `indexOf()` directly as a boolean condition.

Problem example:

```
const fruits = ["apple", "banana", "cherry"];

if (fruits.indexOf("apple")) {
  console.log("Found apple");
}
```

This is dangerous because:

```
fruits.indexOf("apple")
```

returns:

```
0
```

In JavaScript conditions, `0` behaves like false.

Correct version:

```
if (fruits.indexOf("apple") !== -1) {
  console.log("Found apple");
}
```

Better beginner version:

```
if (fruits.includes("apple")) {
  console.log("Found apple");
}
```

Beginner rule:

```
Use includes() for yes/no checks.
Use indexOf() only when you need the position.
```

Part H: `slice()` and `splice()`

`slice()` and `splice()` look similar, but they are very different.

This distinction is important.

23. `slice()` — Copy Part of an Array

`slice()` returns a new array containing part of the original array.

Example:

```
const fruits = ["apple", "banana", "cherry", "date"];

const selected = fruits.slice(1, 3);

console.log(selected); // ["banana", "cherry"]
console.log(fruits);   // ["apple", "banana", "cherry", "date"]
```

The original array is unchanged.

24. How `slice(start, end)` Works

Syntax:

```
array.slice(startIndex, endIndex)
```

The start index is included.

The end index is not included.

Example:

```
const fruits = ["apple", "banana", "cherry", "date"];

console.log(fruits.slice(1, 3));
```

Indexes:

Index:	0	1	2	3
Value:	"apple"	"banana"	"cherry"	"date"

`slice(1, 3)` means:

Start at index 1.
Stop before index 3.

So the result is:

```
["banana", "cherry"]
```

25. `slice()` With One Argument

If you give `slice()` only one argument, it copies from that index to the end.

Example:

```
const fruits = ["apple", "banana", "cherry", "date"];

console.log(fruits.slice(2));
// ["cherry", "date"]
```

26. `slice()` With No Arguments

If you call `slice()` with no arguments, it makes a shallow copy of the whole array.

Example:

```
const fruits = ["apple", "banana", "cherry"];

const copy = fruits.slice();

console.log(copy);
// ["apple", "banana", "cherry"]
```

This is one common way to copy an array.

27. `splice()` — Change the Original Array

`splice()` removes, inserts, or replaces items in the original array.

Example:

```
const fruits = ["apple", "banana", "cherry"];

fruits.splice(1, 1, "blueberry");

console.log(fruits);
// ["apple", "blueberry", "cherry"]
```

Read this:

```
fruits.splice(1, 1, "blueberry");
```

as:

```
Start at index 1.
Delete 1 item.
Insert "blueberry" there.
```

28. How `splice(start, deleteCount, newItems...)` Works

Syntax:

```
array.splice(startIndex, deleteCount, item1, item2, ...)
```

Parts:

Part	Meaning
<code>startIndex</code>	Where to begin changing the array.
<code>deleteCount</code>	How many items to remove.
<code>item1, item2, ...</code>	New items to insert. Optional.

Example 1: remove one item

```
const fruits = ["apple", "banana", "cherry"];

fruits.splice(1, 1);

console.log(fruits);
// ["apple", "cherry"]
```

Example 2: replace one item

```
const fruits = ["apple", "banana", "cherry"];

fruits.splice(1, 1, "blueberry");

console.log(fruits);
// ["apple", "blueberry", "cherry"]
```

Example 3: insert without deleting

```
const fruits = ["apple", "cherry"];

fruits.splice(1, 0, "banana");

console.log(fruits);
// ["apple", "banana", "cherry"]
```

In this example, `deleteCount` is `0`, so nothing is removed.

29. `splice()` Returns Removed Items

`splice()` changes the original array, but it also returns an array of the removed items.

Example:

```
const fruits = ["apple", "banana", "cherry"];

const removed = fruits.splice(1, 1);

console.log(removed); // ["banana"]
console.log(fruits);  // ["apple", "cherry"]
```

30. `slice()` vs `splice()` Summary

Method	Changes Original Array?	Returns	Main Use
<code>slice()</code>	No	New copied array	Copy part of an array
<code>splice()</code>	Yes	Array of removed items	Remove, insert, or replace items

Simple memory trick:

```
slice = copy  
splice = change
```

Beginner warning:

```
Use splice() carefully because it mutates the original array.
```

Part I: Shallow Copies

31. What Is a Shallow Copy?

A shallow copy creates a new array container.

But if the array contains objects or nested arrays, the inner objects are still shared.

Simple meaning:

```
New outer array.  
Same inner objects.
```

32. Shallow Copy With Spread Syntax

You can copy an array with spread syntax:

```
const fruits = ["apple", "banana", "cherry"];  
  
const copy = [...fruits];
```

```
console.log(copy);  
// ["apple", "banana", "cherry"]
```

The spread syntax is:

```
...
```

In this context, it means:

Take the items out of this array and place them into a new array.

33. Shallow Copy With `slice()`

You can also copy an array with `slice()`:

```
const fruits = ["apple", "banana", "cherry"];  
  
const copy = fruits.slice();  
  
console.log(copy);  
// ["apple", "banana", "cherry"]
```

Both of these create a shallow copy:

```
const copy1 = [...fruits];  
const copy2 = fruits.slice();
```

34. Shallow Copies Work Fine for Simple Values

Example:

```
const numbers = [1, 2, 3];  
const copy = [...numbers];  
  
copy[0] = 99;
```

```
console.log(copy);    // [99, 2, 3]
console.log(numbers); // [1, 2, 3]
```

The original array did not change.

This works cleanly because numbers are primitive values.

35. Shallow Copies Share Nested Objects

Example:

```
const users = [{ name: "Ada" }, { name: "Grace" }];

const copy = [...users];

copy[0].name = "Changed";

console.log(users[0].name); // Changed
```

Why did the original change?

Because the array was copied, but the object inside was still shared.

Mental model:

```
users  -> array A -> object 1, object 2
copy   -> array B -> same object 1, same object 2
```

The outer array is new.

The inner objects are the same references.

Beginner rule:

```
Shallow copies protect the outer array.
They do not deeply copy nested objects.
```

Part J: One-Dimensional and Two-Dimensional Arrays

36. One-Dimensional Arrays

A one-dimensional array is a normal list.

Example:

```
const scores = [90, 85, 100];

console.log(scores[0]); // 90
console.log(scores[1]); // 85
console.log(scores[2]); // 100
```

Simple meaning:

A 1D array is one row of values.

Use 1D arrays for simple lists:

```
const tasks = ["read", "code", "review"];
const prices = [10, 20, 30];
const names = ["Ada", "Grace", "Linus"];
```

37. Two-Dimensional Arrays

A two-dimensional array is an array of arrays.

Example:

```
const grid = [
  ["A1", "A2", "A3"],
  ["B1", "B2", "B3"],
  ["C1", "C2", "C3"]
];
```

Think of it like a table:

	Column 0	Column 1	Column 2
Row 0	A1	A2	A3
Row 1	B1	B2	B3
Row 2	C1	C2	C3

38. Reading From a 2D Array

Use two indexes.

Example:

```
const grid = [  
  ["A1", "A2", "A3"],  
  ["B1", "B2", "B3"],  
  ["C1", "C2", "C3"]  
];  
  
console.log(grid[0][0]); // A1  
console.log(grid[1][2]); // B3  
console.log(grid[2][1]); // C2
```

Read this:

```
grid[1][2]
```

as:

Go to row index 1.
Then go to column index 2.

So:

```
grid[1]
```

is:

```
["B1", "B2", "B3"]
```

Then:

```
grid[1][2]
```

is:

```
"B3"
```

Part K: Array Destructuring

39. What Is Array Destructuring?

Array destructuring lets you pull values out of an array into variables.

Example:

```
const colors = ["red", "green", "blue"];

const [first, second, third] = colors;

console.log(first); // red
console.log(second); // green
console.log(third); // blue
```

Simple meaning:

```
Take array items and store them in separate variable names.
```

40. Destructuring Uses Position

Array destructuring is based on position.

Example:

```
const colors = ["red", "green", "blue"];
const [first, second] = colors;
```

```
console.log(first); // red
console.log(second); // green
```

`first` receives index 0.

`second` receives index 1.

The variable names do not have to match anything inside the array.

This works the same:

```
const colors = ["red", "green", "blue"];
const [a, b] = colors;

console.log(a); // red
console.log(b); // green
```

41. Skipping Items During Destructuring

You can skip positions with an extra comma.

Example:

```
const colors = ["red", "green", "blue"];

const [first, , third] = colors;

console.log(first); // red
console.log(third); // blue
```

The empty space between commas skips index 1.

42. Rest Values in Destructuring

You can collect the remaining items with rest syntax:

```
const numbers = [10, 20, 30, 40];

const [first, ...others] = numbers;
```

```
console.log(first); // 10
console.log(others); // [20, 30, 40]
```

The rest syntax is:

```
...
```

In destructuring, it means:

Collect the rest of the values into a new array.

43. Default Values in Destructuring

You can provide a default value if an array item is missing.

Example:

```
const colors = ["red"];

const [first, second = "unknown"] = colors;

console.log(first); // red
console.log(second); // unknown
```

Why?

The array has no index 1.

So `second` uses the default value.

Part L: String Reversal With Array Methods

44. Why Strings and Arrays Connect Here

Strings and arrays both use indexes.

A string:

```
const word = "hello";  
console.log(word[0]); // h
```

An array:

```
const letters = ["h", "e", "l", "l", "o"];  
console.log(letters[0]); // h
```

But strings are immutable.

That means a string cannot be changed in place.

Arrays can be changed.

So one common trick is:

```
Convert string -> array  
Change array  
Convert array -> string
```

45. Reversing a String

Example:

```
const word = "hello";  
  
const reversed = word.split("").reverse().join("");  
  
console.log(reversed); // olleh
```

This uses three methods:

```
split()  
reverse()  
join()
```

46. Step 1: `split("")`

`split("")` turns a string into an array of characters.

Example:

```
const word = "hello";

const letters = word.split("");

console.log(letters);
// ["h", "e", "l", "l", "o"]
```

47. Step 2: `reverse()`

`reverse()` reverses an array in place.

Example:

```
const letters = ["h", "e", "l", "l", "o"];

letters.reverse();

console.log(letters);
// ["o", "l", "l", "e", "h"]
```

Important:

`reverse()` changes the original array.

48. Step 3: `join("")`

`join("")` turns an array back into a string.

Example:

```
const letters = ["o", "l", "l", "e", "h"];
```

```
const word = letters.join("");  
  
console.log(word); // olleh
```

The empty string "" means:

Join the characters with nothing between them.

49. Full String Reversal Dry Run

Code:

```
const word = "hello";  
const reversed = word.split("").reverse().join("");  
console.log(reversed);
```

Step by step:

```
Start:  
"hello"  
  
After split(""):   
["h", "e", "l", "l", "o"]  
  
After reverse():  
["o", "l", "l", "e", "h"]  
  
After join(""):   
"olleh"
```

Expected output:

```
olleh
```

Part M: Core Sprint 6 Code Pattern

50. Core Code Pattern

Type this code yourself.

Before running it, predict the output.

```
const fruits = ["apple", "banana", "cherry"];

console.log(fruits[1]);

fruits[1] = "blueberry";

fruits.push("date");

console.log(fruits);

const reversed = "hello".split("").reverse().join("");

console.log(reversed);

const copy = [...fruits];

console.log(copy);
```

Expected output:

```
banana
["apple", "blueberry", "cherry", "date"]
olleh
["apple", "blueberry", "cherry", "date"]
```

51. Core Code Pattern Explanation

This line creates an array:

```
const fruits = ["apple", "banana", "cherry"];
```

This line reads index 1:

```
console.log(fruits[1]);
```

Because arrays start at 0:

```
index 0 -> apple  
index 1 -> banana  
index 2 -> cherry
```

So it prints:

```
banana
```

This line updates index 1:

```
fruits[1] = "blueberry";
```

Now the array is:

```
["apple", "blueberry", "cherry"]
```

This line adds a new item to the end:

```
fruits.push("date");
```

Now the array is:

```
["apple", "blueberry", "cherry", "date"]
```

This line reverses a string:

```
const reversed = "hello".split("").reverse().join("");
```

It creates:

```
olleh
```

This line creates a shallow copy:

```
const copy = [...fruits];
```

The copy contains the same top-level string values.

Part N: Practical Array Functions

52. Function: `getFirstItem(items)`

Goal:

Return the first item in an array.

```
function getFirstItem(items) {  
  return items[0];  
}  
  
console.log(getFirstItem(["apple", "banana", "cherry"]));
```

Expected output:

```
apple
```

Concepts used:

- function
 - parameter
 - array index
 - return value
-

53. Function: `getLastItem(items)`

Goal:

Return the last item in an array.

```
function getLastItem(items) {  
  return items[items.length - 1];  
}
```

```
console.log(getLastItem(["apple", "banana", "cherry"]));
```

Expected output:

```
cherry
```

Important formula:

```
items.length - 1
```

This gives the last index.

54. Function: `addItem(items, item)`

Goal:

Add an item to the end of an array.

```
function addItem(items, item) {  
  items.push(item);  
  return items;  
}  
  
const tasks = ["read", "code"];  
  
console.log(addItem(tasks, "review"));
```

Expected output:

```
["read", "code", "review"]
```

Important:

This function mutates the original array because `push()` mutates.

55. Function: `hasItem(items, item)`

Goal:

Check if an array contains an item.

```
function hasItem(items, item) {  
  return items.includes(item);  
}  
  
console.log(hasItem(["apple", "banana"], "banana"));  
console.log(hasItem(["apple", "banana"], "grape"));
```

Expected output:

```
true  
false
```

Concepts used:

- function
- parameter
- `includes()`
- boolean return value

56. Function: `replaceItem(items, index, newItem)`

Goal:

Replace an item at a specific index.

```
function replaceItem(items, index, newItem) {  
  items[index] = newItem;  
  return items;  
}  
  
const fruits = ["apple", "banana", "cherry"];  
  
console.log(replaceItem(fruits, 1, "blueberry"));
```

Expected output:

```
["apple", "blueberry", "cherry"]
```

This mutates the original array.

57. Function: `removeItem(items, index)`

Goal:

Remove one item by index.

```
function removeItem(items, index) {  
  items.splice(index, 1);  
  return items;  
}  
  
const fruits = ["apple", "banana", "cherry"];  
  
console.log(removeItem(fruits, 1));
```

Expected output:

```
["apple", "cherry"]
```

This uses `splice()`.

It mutates the original array.

58. Function: `copyItems(items)`

Goal:

Return a shallow copy of an array.

```
function copyItems(items) {  
  return [...items];  
}  
  
const original = ["read", "code", "review"];  
const copy = copyItems(original);  
  
console.log(copy);
```

Expected output:

```
["read", "code", "review"]
```

Alternative version:

```
function copyItems(items) {  
  return items.slice();  
}
```

Part O: Mini-Build — Todo Array

59. Mini-Build Goal

Create a small todo list using an array.

You will write functions to:

```
Create a todo  
Read todos  
Update a todo  
Delete a todo
```

This is a simple version of CRUD.

CRUD means:

```
Create  
Read  
Update  
Delete
```

60. Todo Array Code

```
const todos = ["study arrays", "practice functions"];  
  
function addTodo(todo) {  
  todos.push(todo);  
}
```

```
function readTodos() {  
  return todos;  
}  
  
function updateTodo(index, newTodo) {  
  todos[index] = newTodo;  
}  
  
function deleteTodo(index) {  
  todos.splice(index, 1);  
}  
  
addTodo("build a mini project");  
updateTodo(0, "study JavaScript arrays");  
deleteTodo(1);  
  
console.log(readTodos());
```

Expected output:

```
["study JavaScript arrays", "build a mini project"]
```

61. Todo Array Explanation

Initial array:

```
const todos = ["study arrays", "practice functions"];
```

This function adds a todo:

```
function addTodo(todo) {  
  todos.push(todo);  
}
```

It uses `push()` to add to the end.

This function reads the todos:


```
function readTodos() {  
  return todos;  
}
```

It returns the array.

This function updates a todo:

```
function updateTodo(index, newTodo) {  
  todos[index] = newTodo;  
}
```

It uses index assignment.

This function deletes a todo:

```
function deleteTodo(index) {  
  todos.splice(index, 1);  
}
```

It uses `splice()` to remove one item.

62. Todo Array Dry Run

Start:

```
["study arrays", "practice functions"]
```

After:

```
addTodo("build a mini project");
```

Array becomes:

```
["study arrays", "practice functions", "build a mini project"]
```

After:

```
updateTodo(0, "study JavaScript arrays");
```

Array becomes:

```
["study JavaScript arrays", "practice functions", "build a mini project"]
```

After:

```
deleteTodo(1);
```

The item at index 1 is removed.

Array becomes:

```
["study JavaScript arrays", "build a mini project"]
```

Part P: Mutation vs Copying

63. What Is Mutation?

Mutation means changing the original array.

These methods mutate the original array:

```
push()  
pop()  
unshift()  
shift()  
splice()  
reverse()
```

Example:

```
const fruits = ["apple", "banana"];  
  
fruits.push("cherry");
```

```
console.log(fruits);  
// ["apple", "banana", "cherry"]
```

The original array changed.

64. What Is Copying?

Copying means creating a new array instead of changing the original.

These can create copies:

```
slice()  
spread syntax [...array]
```

Example:

```
const fruits = ["apple", "banana"];  
  
const copy = [...fruits];  
  
console.log(copy);  
// ["apple", "banana"]
```

65. Why Mutation Matters

Mutation is not always bad.

Many beginner examples use mutation because it is simple.

But mutation matters because other parts of your code may be using the same array.

Example:

```
const original = ["a", "b", "c"];  
const sameArray = original;  
  
sameArray.push("d");
```

```
console.log(original);  
// ["a", "b", "c", "d"]
```

Why?

Both variables point to the same array.

This connects to the reference concept from Sprint 2.

Beginner rule:

If you need to preserve the original array, make a copy first.

Example:

```
const original = ["a", "b", "c"];  
const copy = [...original];  
  
copy.push("d");  
  
console.log(original); // ["a", "b", "c"]  
console.log(copy);     // ["a", "b", "c", "d"]
```

Part Q: Common Sprint 6 Mistakes

66. Mistake 1: Thinking Index Starts at 1

Wrong mental model:

```
fruits[1] // first fruit
```

Correct mental model:

```
fruits[0] // first fruit
```

Example:

```
const fruits = ["apple", "banana", "cherry"];

console.log(fruits[0]); // apple
```

67. Mistake 2: Thinking `.length` Is the Last Index

Wrong:

```
const fruits = ["apple", "banana", "cherry"];

console.log(fruits[fruits.length]); // undefined
```

Correct:

```
console.log(fruits[fruits.length - 1]); // cherry
```

Reason:

```
length = 3
last index = 2
```

68. Mistake 3: Confusing `slice()` and `splice()`

Wrong assumption:

```
slice() and splice() do almost the same thing.
```

Correct:

```
slice() copies.
splice() changes.
```

Example:

```
const fruits = ["apple", "banana", "cherry"];

const copy = fruits.slice(1);
console.log(copy);    // ["banana", "cherry"]
console.log(fruits);  // ["apple", "banana", "cherry"]
```

But:

```
const fruits = ["apple", "banana", "cherry"];

fruits.splice(1, 1);
console.log(fruits); // ["apple", "cherry"]
```

69. Mistake 4: Thinking `const` Freezes the Array

Wrong idea:

```
A const array cannot be changed at all.
```

Correct idea:

```
A const array cannot be reassigned, but its contents can change.
```

Allowed:

```
const fruits = ["apple"];
fruits.push("banana");

console.log(fruits);
// ["apple", "banana"]
```

Not allowed:

```
const fruits = ["apple"];
fruits = ["grape"]; // error
```

70. Mistake 5: Forgetting That `push()` Returns Length

Example:

```
const fruits = ["apple"];

const result = fruits.push("banana");

console.log(result); // 2
```

Some beginners expect `result` to be the array.

But `push()` returns the new length.

The array itself becomes:

```
["apple", "banana"]
```

71. Mistake 6: Forgetting That `pop()` and `shift()` Return Removed Items

Example:

```
const fruits = ["apple", "banana", "cherry"];

const removed = fruits.pop();

console.log(removed); // cherry
console.log(fruits); // ["apple", "banana"]
```

`pop()` returns the removed item.

Same idea with `shift()`:

```
const fruits = ["apple", "banana", "cherry"];

const removed = fruits.shift();
```

```
console.log(removed); // apple
console.log(fruits); // ["banana", "cherry"]
```

72. Mistake 7: Thinking a Shallow Copy Deeply Copies Everything

Problem example:

```
const users = [{ name: "Ada" }];
const copy = [...users];

copy[0].name = "Grace";

console.log(users[0].name); // Grace
```

Why?

The array was copied, but the object inside was shared.

Correct understanding:

A shallow copy copies the outer array only.
Nested objects are still shared.

Part R: Practice Set

73. Warm-Up Drill 1: Create Arrays

Create three arrays:

```
const names = ["Ada", "Grace", "Linus"];
const scores = [90, 85, 100];
const tasks = ["read", "code", "review"];

console.log(names);
console.log(scores);
console.log(tasks);
```

Goal:

Practice array syntax.

74. Warm-Up Drill 2: Read Array Items

```
const fruits = ["apple", "banana", "cherry"];

console.log(fruits[0]);
console.log(fruits[1]);
console.log(fruits[2]);
console.log(fruits[99]);
```

Expected output:

```
apple
banana
cherry
undefined
```

Question:

Why is the last output `undefined`?

Answer:

There is no item at index 99.

75. Warm-Up Drill 3: Use `.length`

```
const tasks = ["read", "code", "review"];

console.log(tasks.length);
console.log(tasks[tasks.length - 1]);
```

Expected output:

```
3
review
```

76. Warm-Up Drill 4: Add and Remove Items

```
const tasks = ["read", "code"];

tasks.push("review");
tasks.unshift("plan");

console.log(tasks);

const last = tasks.pop();
const first = tasks.shift();

console.log(last);
console.log(first);
console.log(tasks);
```

Expected output:

```
["plan", "read", "code", "review"]
review
plan
["read", "code"]
```

77. Warm-Up Drill 5: Find Items

```
const fruits = ["apple", "banana", "cherry"];

console.log(fruits.includes("banana"));
console.log(fruits.includes("grape"));
console.log(fruits.indexOf("cherry"));
console.log(fruits.indexOf("grape"));
```

Expected output:

```
true
false
2
-1
```

78. Warm-Up Drill 6: Practice `slice()` and `splice()`

```
const fruits = ["apple", "banana", "cherry", "date"];

const selected = fruits.slice(1, 3);

console.log(selected);
console.log(fruits);

fruits.splice(1, 1, "blueberry");

console.log(fruits);
```

Expected output:

```
["banana", "cherry"]
["apple", "banana", "cherry", "date"]
["apple", "blueberry", "cherry", "date"]
```

Question:

Which method changed the original array?

Answer:

`splice()` changed the original array.

79. Warm-Up Drill 7: Shallow Copy

```
const original = ["read", "code", "review"];
const copy = [...original];

copy.push("practice");

console.log(original);
console.log(copy);
```

Expected output:

```
["read", "code", "review"]  
["read", "code", "review", "practice"]
```

Question:

Why did the original not change?

Answer:

The copied array is a different outer array.

80. Warm-Up Drill 8: 2D Array

```
const grid = [  
  ["A1", "A2", "A3"],  
  ["B1", "B2", "B3"],  
  ["C1", "C2", "C3"]  
];  
  
console.log(grid[0][0]);  
console.log(grid[1][2]);  
console.log(grid[2][1]);
```

Expected output:

```
A1  
B3  
C2
```

81. Warm-Up Drill 9: Array Destructuring

```
const colors = ["red", "green", "blue"];  
  
const [first, second, third] = colors;  
  
console.log(first);  
console.log(second);  
console.log(third);
```

Expected output:

```
red  
green  
blue
```

82. Warm-Up Drill 10: Reverse a String

```
const word = "hello";  
const reversed = word.split("").reverse().join("");  
  
console.log(reversed);
```

Expected output:

```
olleh
```

Part S: Debug Challenges

83. Debug Challenge 1: Missing Index

Code:

```
const fruits = ["apple", "banana", "cherry"];  
  
console.log(fruits[5]);
```

Task:

Predict the output.

Expected output:

```
undefined
```

Explanation:

The array has indexes 0, 1, and 2.

There is no index 5.

84. Debug Challenge 2: Last Item Mistake

Broken code:

```
const fruits = ["apple", "banana", "cherry"];

console.log(fruits[fruits.length]);
```

Problem:

`fruits.length` is `3`, but the last valid index is `2`.

Correct code:

```
console.log(fruits[fruits.length - 1]);
```

Expected output:

```
cherry
```

85. Debug Challenge 3: `slice()` vs `splice()`

Code:

```
const fruits = ["apple", "banana", "cherry"];

const result = fruits.slice(1, 2);

console.log(result);
console.log(fruits);
```

Expected output:

```
["banana"]  
["apple", "banana", "cherry"]
```

Explanation:

`slice()` does not change the original array.

86. Debug Challenge 4: `push()` Return Value

Code:

```
const fruits = ["apple"];  
  
const result = fruits.push("banana");  
  
console.log(result);  
console.log(fruits);
```

Expected output:

```
2  
["apple", "banana"]
```

Explanation:

`push()` returns the new length.

It also changes the original array.

Part T: Full Practice Assignment

87. Practice Assignment Code

Write this from memory:

```
const tasks = ["read", "code", "review"];  
  
tasks.push("practice");  
tasks[1] = "write code";
```

```
console.log(tasks[0]);
console.log(tasks[tasks.length - 1]);
console.log(tasks.includes("review"));
console.log(tasks.indexOf("practice"));

const copiedTasks = tasks.slice();

tasks.splice(0, 1);

console.log(tasks);
console.log(copiedTasks);
```

88. Practice Assignment Expected Output

Step by step:

Start:

```
["read", "code", "review"]
```

After:

```
tasks.push("practice");
```

Array becomes:

```
["read", "code", "review", "practice"]
```

After:

```
tasks[1] = "write code";
```

Array becomes:

```
["read", "write code", "review", "practice"]
```

These lines:


```
console.log(tasks[0]);  
console.log(tasks[tasks.length - 1]);  
console.log(tasks.includes("review"));  
console.log(tasks.indexOf("practice"));
```

print:

```
read  
practice  
true  
3
```

Then:

```
const copiedTasks = tasks.slice();
```

creates a shallow copy.

Then:

```
tasks.splice(0, 1);
```

removes the item at index 0.

Final arrays:

```
tasks  
// ["write code", "review", "practice"]  
  
copiedTasks  
// ["read", "write code", "review", "practice"]
```

Full expected output:

```
read  
practice  
true  
3
```

```
["write code", "review", "practice"]  
["read", "write code", "review", "practice"]
```

89. Practice Assignment Questions

Answer these after running the code:

1. Which operations changed the original array?
2. Which method made a copy?
3. Why did `tasks[0]` change after `splice()`?
4. Why did `copiedTasks` keep the original values?
5. What is the last valid index before `splice()` runs?
6. What does `tasks.includes("review")` return?
7. What does `tasks.indexOf("practice")` return?

Suggested answers:

1. `push()`, index assignment, and `splice()` changed the original array.
2. `slice()` made a copy.
3. `splice(0, 1)` removed the first item, so the next item moved into index 0.
4. `copiedTasks` was created before `splice()` changed `tasks`.
5. Before `splice()`, the last valid index is `3`.
6. It returns `true`.
7. It returns `3` before `splice()`.

Part U: Sprint 6 Revision Checkpoint

90. Core Questions

Answer these without looking at the notes.

1. What is an array?
2. What does zero-indexed mean?
3. What is the first index in an array?
4. What does `array.length` return?
5. Why is the last index `array.length - 1`?
6. What happens when you access a missing index?
7. Why can you change items inside a `const` array?
8. What does `push()` do?
9. What does `pop()` do?
10. What does `unshift()` do?
11. What does `shift()` do?

12. What does `includes()` return?
 13. What does `indexOf()` return when the item is not found?
 14. What is the difference between `slice()` and `splice()`?
 15. What is a shallow copy?
 16. Why do nested objects stay shared in a shallow copy?
 17. What is a 2D array?
 18. How do you read an item from a 2D array?
 19. What does array destructuring do?
 20. How can you reverse a string using array methods?
-

91. Short Answers

1. An array is an ordered list of values.
 2. Zero-indexed means the first item is at index 0.
 3. The first index is 0.
 4. `array.length` returns the number of items.
 5. Because indexes start at 0, not 1.
 6. JavaScript returns `undefined`.
 7. `const` prevents reassignment, not internal mutation.
 8. `push()` adds to the end and returns the new length.
 9. `pop()` removes from the end and returns the removed item.
 10. `unshift()` adds to the beginning and returns the new length.
 11. `shift()` removes from the beginning and returns the removed item.
 12. `includes()` returns `true` or `false`.
 13. `indexOf()` returns `-1` when the item is not found.
 14. `slice()` copies; `splice()` changes.
 15. A shallow copy is a new outer array with the same top-level elements.
 16. Nested objects are references, so the copied array still points to the same inner objects.
 17. A 2D array is an array of arrays.
 18. Use two indexes, such as `grid[1][2]`.
 19. Destructuring pulls array values into variables.
 20. Use `split("").reverse().join("")`.
-

Part V: Sprint 6 Mastery Checklist

You are ready for Sprint 7 when you can do all of these without looking:

- ☐ Create an array.
- ☐ Explain what an ordered collection means.
- ☐ Read the first item with index 0.
- ☐ Read the last item with `array.length - 1`.
- ☐ Explain zero-based indexing.

- [] Explain why a missing index returns undefined.
- [] Use .length correctly.
- [] Update an item by index.
- [] Explain why const arrays can still be changed inside.
- [] Use push() to add to the end.
- [] Use pop() to remove from the end.
- [] Use unshift() to add to the beginning.
- [] Use shift() to remove from the beginning.
- [] Explain what each add/remove method returns.
- [] Use includes() for true/false existence checks.
- [] Use indexOf() to find an item position.
- [] Explain why indexOf() returns -1 when not found.
- [] Use slice() without mutating the original array.
- [] Use splice() to remove, insert, or replace an item.
- [] Explain the difference between slice() and splice().
- [] Make a shallow copy with spread syntax.
- [] Make a shallow copy with slice().
- [] Explain why shallow copies still share nested objects.
- [] Create and read a 1D array.
- [] Create and read a 2D array.
- [] Use array destructuring.
- [] Skip an item in destructuring.
- [] Use rest syntax in destructuring.
- [] Use a default value in destructuring.
- [] Reverse a string with split(), reverse(), and join().
- [] Build a simple todo array with add, read, update, and delete functions.

Part W: Final Sprint 6 Summary

92. The Most Important Ideas

An array is a list:

```
const fruits = ["apple", "banana", "cherry"];
```

Arrays are ordered:

```
index 0 -> apple
index 1 -> banana
index 2 -> cherry
```

Arrays start at index 0:

```
fruits[0]; // apple
```

`.length` counts items:

```
fruits.length; // 3
```

The last index is length minus 1:

```
fruits[fruits.length - 1]; // cherry
```

You can update by index:

```
fruits[1] = "blueberry";
```

You can add and remove items:

```
fruits.push("date");  
fruits.pop();  
fruits.unshift("orange");  
fruits.shift();
```

You can search arrays:

```
fruits.includes("apple");  
fruits.indexOf("banana");
```

You can copy parts with `slice()`:

```
const selected = fruits.slice(1, 3);
```

You can change the original with `splice()`:

```
fruits.splice(1, 1, "blueberry");
```

You can make shallow copies:

```
const copy1 = [...fruits];  
const copy2 = fruits.slice();
```

You can use 2D arrays for grid-like data:

```
const grid = [  
  ["A1", "A2"],  
  ["B1", "B2"]  
];  
  
console.log(grid[1][0]); // B1
```

You can destructure arrays:

```
const [first, second] = fruits;
```

You can reverse a string through array methods:

```
const reversed = "hello".split("").reverse().join("");
```

Final beginner rule:

Use arrays when you need to store many related values in order.
Learn which methods mutate the original array and which methods make copies.